

Basic Network Sniffer

Name:- **Arwa Ajmeri**

Internship

Cyber Security Internship – Horizon TechX

Project Title

Basic Network Sniffer

Objective

The objective of this project is to understand how network packets travel across a network by capturing and analyzing live packets using Python and the Scapy library.

Tools Used

- Python 3.14
- Visual Studio Code
- Scapy Library
- Npcap
- Windows 10

Project Description

This project captures live network packets and displays useful information such as:

- Packet Number
- Source IP Address
- Destination IP Address
- Protocol (TCP/UDP/ICMP)

It helps in understanding basic networking concepts and packet analysis.

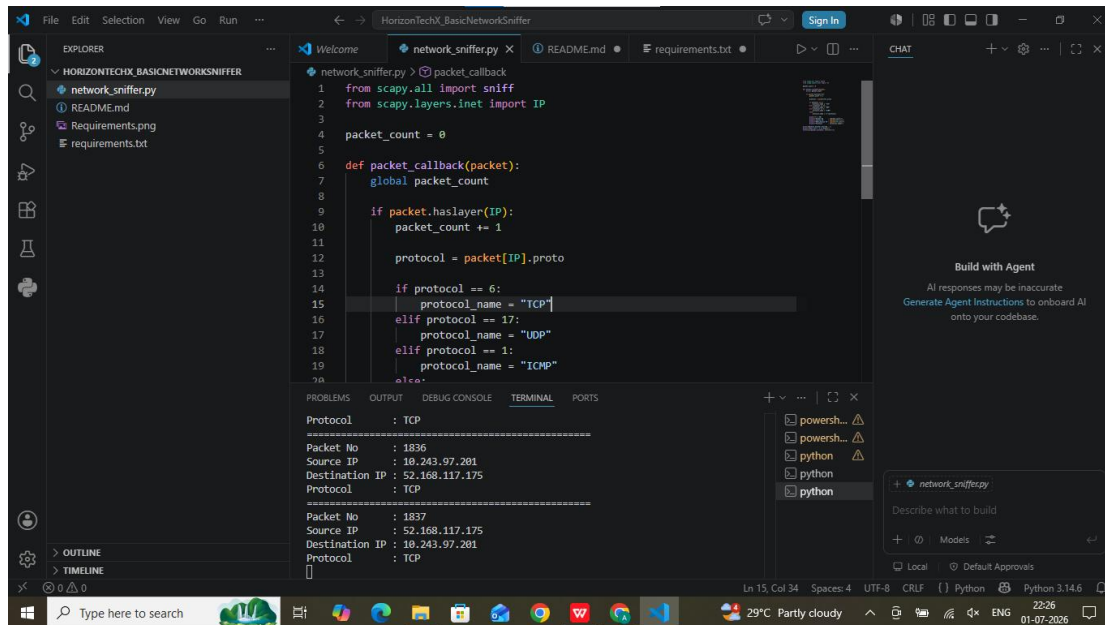
Output

Paste your **output.png** screenshot here.

Conclusion

This project successfully captured live network packets and displayed Source IP, Destination IP, and Protocol information. It improved my understanding of network communication and packet analysis using Python.

1. NETWORK SNIFFER



The screenshot shows the Visual Studio Code editor with a project named 'HorizonTechX_BasicNetworkSniffer'. The file explorer on the left shows 'network_sniffer.py', 'README.md', 'Requirements.png', and 'requirements.txt'. The main editor displays the 'network_sniffer.py' script, which uses Scapy to sniff network packets. The script defines a 'packet_callback' function that increments a counter and identifies the protocol (TCP, UDP, ICMP) based on the packet's protocol number. The terminal window at the bottom shows the output of the script, displaying two captured packets: a TCP packet from 10.243.97.281 to 52.168.117.175 and another TCP packet from 52.168.117.175 to 10.243.97.281.

```
1 from scapy.all import sniff
2 from scapy.layers.inet import IP
3
4 packet_count = 0
5
6 def packet_callback(packet):
7     global packet_count
8
9     if packet.haslayer(IP):
10         packet_count += 1
11
12         protocol = packet[IP].proto
13
14         if protocol == 6:
15             protocol_name = "TCP"
16         elif protocol == 17:
17             protocol_name = "UDP"
18         elif protocol == 1:
19             protocol_name = "ICMP"
20         else:
21             protocol_name = "Unknown"
```

Protocol : TCP

Packet No : 1836

Source IP : 10.243.97.281

Destination IP : 52.168.117.175

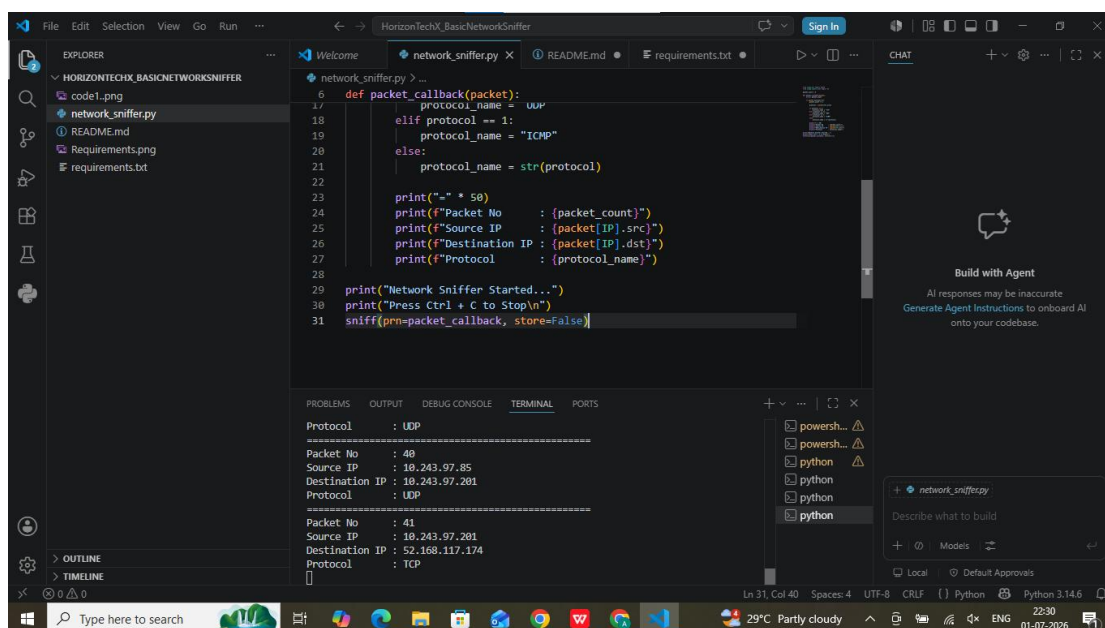
Protocol : TCP

Packet No : 1837

Source IP : 52.168.117.175

Destination IP : 10.243.97.281

Protocol : TCP



The screenshot shows the Visual Studio Code editor with the same project. The 'network_sniffer.py' script is now running, and the terminal window displays the output of the script. The script has been modified to print detailed information about each captured packet, including the packet number, source IP, destination IP, and protocol. The terminal shows two captured packets: a UDP packet from 10.243.97.85 to 10.243.97.281 and another UDP packet from 10.243.97.281 to 52.168.117.174.

```
17 def packet_callback(packet):
18     protocol_name = "UDP"
19     elif protocol == 1:
20         protocol_name = "ICMP"
21     else:
22         protocol_name = str(protocol)
23
24     print(f"Packet No : {packet[IP].no}")
25     print(f"Source IP : {packet[IP].src}")
26     print(f"Destination IP : {packet[IP].dst}")
27     print(f"Protocol : {protocol_name}")
28
29 print("Network Sniffer Started...")
30 print("Press Ctrl + C to Stop")
31 sniff(prn=packet_callback, store=False)
```

Protocol : UDP

Packet No : 40

Source IP : 10.243.97.85

Destination IP : 10.243.97.281

Protocol : UDP

Packet No : 41

Source IP : 10.243.97.281

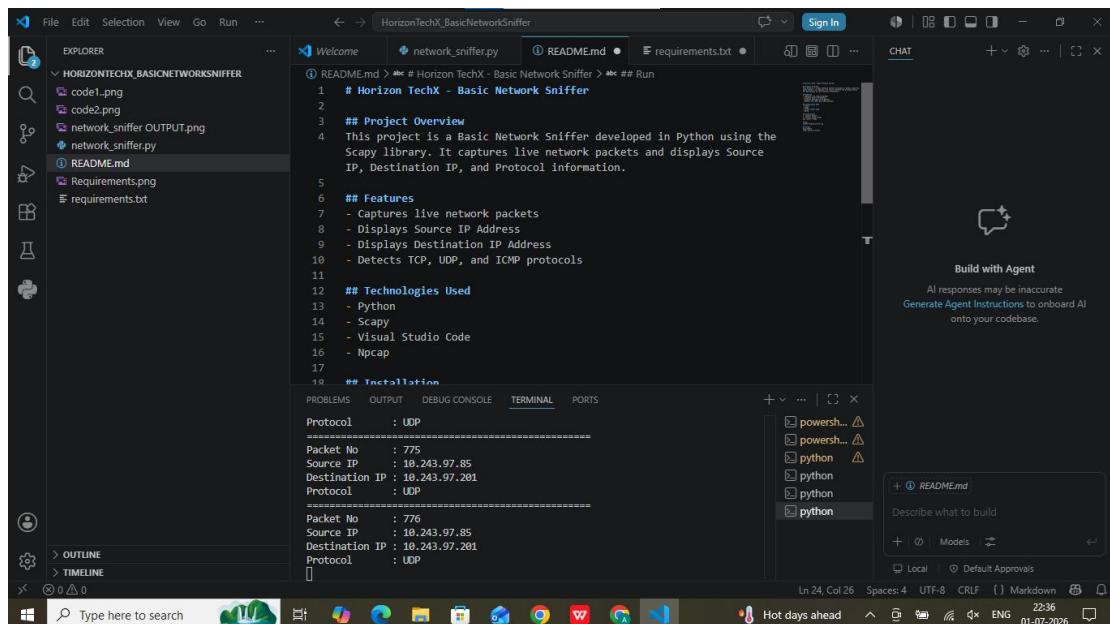
Destination IP : 52.168.117.174

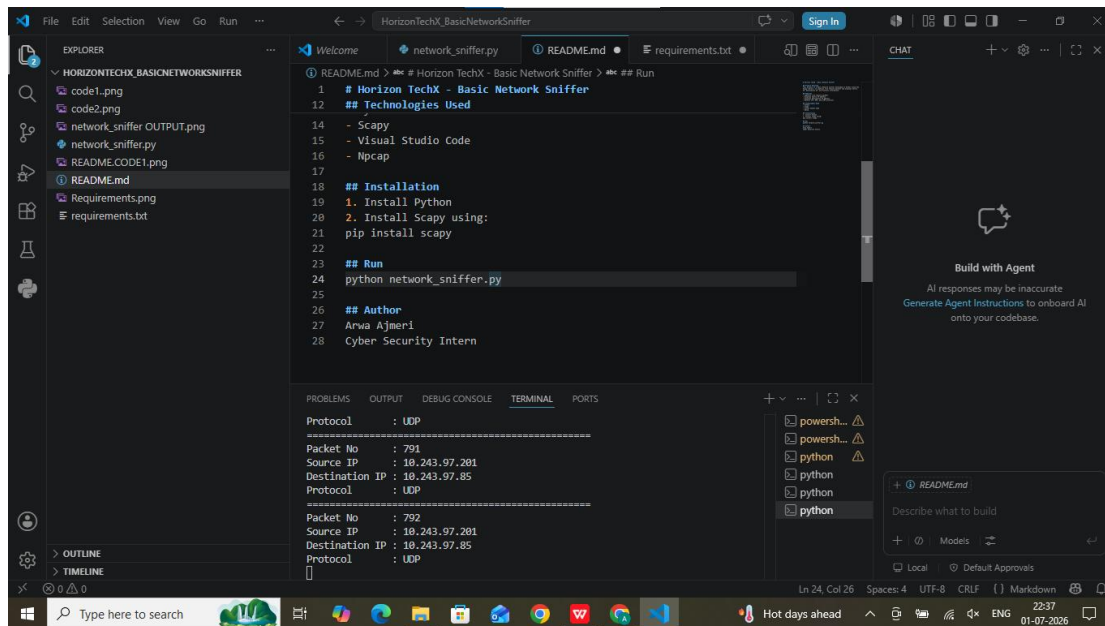
Protocol : TCP

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Protocol      : UDP
=====
Packet No     : 47
Source IP     : 10.243.97.201
Destination IP : 10.243.97.255
Protocol      : UDP
=====
Packet No     : 48
Source IP     : 10.243.97.201
Destination IP : 10.243.97.255
Protocol      : UDP
█
```

2. README.MD





```
Protocol      : UDP
=====
Packet No     : 810
Source IP     : 10.243.97.85
Destination IP : 10.243.97.201
Protocol      : UDP
=====
Packet No     : 811
Source IP     : 10.243.97.85
Destination IP : 10.243.97.201
Protocol      : UDP
□
```

3. Requirements

